Assignment 3: "K In A Row" -- Agents and Tournament"

CSE 415: Introduction to Artificial Intelligence
The University of Washington, Seattle, Autumn 2026

Due Wednesday, Oct. 21, at 11:59 PM. Partnerships are optional for this assignment but strongly encouraged. Complete the planning questionnaire anytime between Friday, Oct. 16 and Wednesday night, Oct. 20, for 1 point of participation credit. Earlier is better. For example, partnering can become more difficult as time goes on. (See the link to the questionnaire at the bottom of this page.)

MODES OF PLAY: Your agent should support three modes of play: (A) Demo: Demonstration Mode, in which time used is not critical but should not take longer than 3 sec. per move. The agent will carry on a conversation during the game. Optionally, your code may import an API for an LLM to assist with utterance generation. However, the staff might be unable to run your agent in this mode, due to a missing API key in the environment. (B) Autograder: In this mode, specific features will be enabled and disabled by the autograder to test your agent's functionality and assign points. (C) Competition: In this mode, the conversation will be turned off, except for each agent introducing itself. Timing will be important, to enable its use as a basis for elimination in some or all of the competition rounds. In this mode, your agent must not import any modules that require additional installations beyond the standard Python distribution from Python.org.

Student starter code is available for this assignment.

A. Introduction

K-in-a-Row

In this assignment, you'll create a program that implements an agent that can participate in a game of K-in-a-Row with Forbidden Squares (defined below). Then you'll demonstrate your agent, providing one or more transcripts of the agent playing against another agent.

PART I: Agent Code to Play K-in-a-Row (20 points). Your program should consist of a single file with a name like **`yourID_KInARow.py`** where the first author's ID is used.

Your program will consist mainly of a sub-class OurAgent of KAgent, providing collection of specific methods, for playing games of "K in a Row". We define a K in a Row game as a kind of generalized Tic-Tac-Toe with the following features:

- (a) Just as in Tic-Tac-Toe, there are two players: one plays X and the other plays O;
- (b) the board is rectangular, but is not necessarily 3 by 3; it is nRows by mColumns, where these numbers are set by the Game Master (referee) at the

beginning of the game according to the current game type (e.g., Tic-Tac-Toe, Five-in-a-Row, Cassini);

- (c) a player wins by getting K in a row, where K is not necessarily 3; K can be any integer greater than 1 and less than or equal to the maximum of mRows and nColumns;
- (d) there can be "forbidden squares" on the board; these are chosen at the beginning of the game by the Game Master; a square on the board that is available is represented by a blank, whereas a forbidden square is represented by a dash "-" ;
- (e) there can be "handicaps" in the initial state, meaning that some X and/or O tokens can be set up on the board by the Game Master in order either to influence the succeeding play or to change the balance of advantage and disadvantage to the players. For example, the initial state of Cassini has two Os already on the board.

The starter code consists of several files:

File you will modify and turn in:

- **`yourID_KInARow.py`** (rename it as explained above)

File you will modify and not turn in:

- **`Game_Master_Offline.py`** (edit the test function at the end to include your agent and choose a game)

Files you will create and turn in:

- GameTranscript**`.pdf`** An HTML file will be created and named automatically by the Game Master. It will represent the transcript of a match between your agent and that of another team. It will be in the form of an html page that can be viewed in a browser. please keep the file name but convert the file to PDF for submission. One option for this is to use pandoc. Another is to run a local webserver from the command line in the folder containing your .html file with this command: **`python -m http.server 5000`**. Then open a web browser with the URL **`http://localhost:5000`** and then use your browser's print option to print to PDF.
- **`Learning-Diary-Entry.pdf`** — Your completed Learning Diary entry for this assignment. In this assignment, instead of an "Assignment Report" you will submit something similar, but it is called a "Learning Diary entry" here. Use the Learning Diary Entry template (in the starter code) as your starting point. It covers your agent design, search algorithm analysis, dialog design, gen-AI use and attribution, and any extra credit items.

Files you should read and understand:

- **`game_types.py`** — defines a State class for all the games, and a Game_Type class for the games. *As you read: Where is **`k`** (the win length) stored? What do the tokens **`' '`**, **`'-'`**, **`'X'`**, and **`'O'`** each mean? How do you make a deep copy of a state before modifying it?*
- **`agent_base.py`** — defines a basic agent that you'll subclass. *As you read: What are the three playing modes (DEMO, COMPETITION, AUTOGRADER) and when does each apply? What must **`prepare`** return?*

What four statistics go in the AUTOGRADER return list, and in what order?

- **`RandomPlayer.py`** — a sample agent that plays randomly. *As you read: How does it generate the list of legal moves? How does it implement the twin feature? What does its **`prepare`** method save for later use?*

Other files:

- **`winTesterForK.py`** (helper to Game_Master_Offline.py)
- **`gameToHTML.py`** (creates the HTML page transcript of the game session)
- **`gameToJSON.py`** (creates a JSON transcript of the same game, alongside the HTML one). A JSON transcript can be replayed later — including in a browser, with the agents' utterances spoken aloud — so your instructor may ask you to hand in the **`.json`** file as well as the PDF. Both recorders can be switched off at the top of **`Game_Master_Offline.py`**; both are on by default.
- **`spec_static_by_table.py`** (the "special" static evaluation function used in AUTOGRADER mode; see Part I)
- **`autograder.py`** — a self-check autograder you can run yourself, before submitting, with **`python3 autograder.py`**. It exercises the same Part I tests described below (agent loading, **`prepare`** and **`introduce`**, a full game play-through, static evaluation ordering, minimax, and alpha-beta pruning) and prints the points it would award. Edit the **`AGENT_FILE`** line at the top to match your agent's file name.
- **`X32.png`**, **`O32.png`**, **`black32.png`**, **`gray32.png`** (board images used in the HTML transcripts)

Key Methods to Implement

Some of the functionality you need to implement in your agent is templated or given simple versions in the starter code. For example:

```
prepare(self, game_type, what_side_to_play, opponent_nickname,
        expected_time_per_move=0.1, utterances_matter=True)
```

Called by the Game Master before play begins. Use it to store the game type, your side ('X' or 'O'), and any other information your agent needs across turns. **Your `prepare` method must return the string `'OK'`** to signal successful initialization; the autograder checks for this exact value before running any tests. If **`utterances_matter`** is **`False`** (competition mode), skip importing any LLM APIs and return a simple utterance each turn.

```
introduce(self)
```

This function should return a multi-line string to introduce your agent. This should include your agent's name (you get to make this up), your name and student ID, and anything else you'd like the agent to say at the start of the game.

```
make_move(self, state, current_utterance, time_limit,
          use_alpha_beta=True,
          use_zobrist_hashing=False, max_ply=3,
          special_static_eval_fn=None)
```

This function determines which move to make given a game board (state). It is the main entry point into your game playing algorithm, and is called by the Game

Master.

If the **time_limit** argument is not **None**, the Game Master will cut off your agent's execution once that many seconds have passed. If you have not returned a valid move by that time, you will forfeit the game. To make sure this does not happen, you can handle the **time_limit** parameter to end your search early and return your best move so far.

You may assume that the **state** provided to this function will be a valid state where there is a possible space to move and no winner has been established.

In DEMO and COMPETITION modes the method should return **[[newMove, newState], myUtterance]**, where **newMove** is a pair **(i, j)** giving the row and column where the player places an X or O, **newState** is the resulting State object, and **myUtterance** is the player's dialog string for this turn — at least one word, typically one sentence, "in character" with your agent's persona. (See "Utterances" below.)

In AUTOGRADER mode, append a statistics dictionary as a third element: **[[newMove, newState, stats_dict], myUtterance]**. The dictionary keys and their default values are defined in the **make_move** stub in **agent_base.py**; fill them in with measured values so the autograder can score your alpha-beta and Zobrist implementations.

Your minimax function and alpha-beta pruning will be graded, in-part, based on the number of cutoffs they make in a controlled game-tree search. In AUTOGRADER mode, the autograder's "special" static evaluation function has to be used by your agent. Your move-generator should consider moves in a specific order when in this mode -- the same order that they are generated (but the order not in which they are selected, which is where the randomness comes in) by the RandomPlayer agent. Also in autograder mode, it will be important to respect the max-ply value that is passed in.

The parameter **use_zobrist_hashing** can be used by the autograder to specify whether or not this feature (optionally implemented) is turned on or off for a comparative test of its effects. If you implement Zobrist hashing, and it works well, you should have it on by default, and whether turned on or off, return valid statistics. If you do not implement Zobrist hashing, your agent can ignore the parameter and just return the default stats (-1, for example) that are in specified in the agent_base starter code.

```
minimax(self, state, depth_remaining, pruning=False,
alpha=None, beta=None)
```

This function should be called by **make_move** and should implement the minimax algorithm. **state** and **depth_remaining** should be used for recursively searching deeper layers and stopping at the correct layer, and will be used in our testing. The remaining arguments are up to you to use how you wish, with the caveat that your code must be able to handle a value of **None** for any of the remaining arguments which indicates to not use the associated feature.

The parameters **alpha** and **beta** are present to support the implementation of your agent's alpha-beta pruning. The minimax method should be coordinated with your make_move method. For example in make_move, the autograder might pass in a value of use_alpha_beta=False; in that case your minimax method should ignore the values of parameters alpha and beta.

The minimax function must return a move and a float value that corresponds to the evaluation of that move.

`static_eval(self, state, game_type=None):`

This is your static evaluation function. Given a game state, it should return a numeric evaluation of that state where a higher value means that the state is better for X and a lower value is a state better for O. Note that this is regardless of which piece your agent is playing. For this assignment we define that X will always be the maximizing player and O always the minimizing player.

In normal game-play, your agent will know what type of game is being played, because the game master will provide that information via the call to your agent's **`prepare`** method. However, during autograding, your static evaluation function might be called with not only a state, but a game type. Although one could sometimes guess the game type from the state, the actual value of k is not necessarily specified in the state. If your static evaluation function were given a 10 by 15 board with some Xs and Os on it, it would be difficult to evaluate without knowing, say, that k is 9 in the game. So the game_type argument would provide the information about k as well as the other properties of the game.

Your static evaluation function should be non-trivial and should return different evaluations for states that are clearly different. We will grade based on comparing your evaluations across states that are easily distinguished by a human as significantly better or worse, so don't worry too much about determining small differences between mostly even states.

The amount of work done by your static evaluation function should depend on the type of game being played, which will be stored as a property of your agent. The game type has the value of k, which is the number of tokens in a line required to win the game. Your static evaluation function will probably need to make use of this value k. For example, if there is a line of k-1 Xs and O has not blocked the completion of k in a row, then this state should probably have a very high value.

We provide little guidance for this function beyond the lecture slides. This will be the main opportunity for you to set your agent apart from the rest of the class for the tournament, so making sure that this function gives good results and runs quickly will go a long ways towards improving your agent's performance.

When a call to make_move happens in AUTOGRADER mode, then your code should use the special static evaluation function given in the call instead of your own function. When not in AUTOGRADER mode, then no special static evaluation function will be provided in the call.

For our testing, you may assume that the **`state`** parameter is a valid state that matches the size and k values of the initial state passed to your agent in the **`prepare`** method call.

PART II: Agent Code for Dialog (20 points).

Utterances:

Each player participates not only in the game being played, but also in a conversation that takes place at the same time. In each turn, the player returns both a move (and corresponding new game state) and a text string we call the utterance. The utterance should be "in character" with your agent's persona.

Ideally, it is also somewhat intelligent in that it represents comments that are appropriate. The utterances may achieve any subset of the following behaviors: persona-specific, game-specific, game-state-specific, funny, serious, observant, didactic (teaching something about the agent technology or search techniques), or responsiveness to the utterances of the opponent.

We expect each team or individual to make some effort to incorporate interesting technology or creativity into the production of the utterances. Possibilities include: (a) using an LLM with well-designed prompting to achieve both the desired persona and appropriate comments on the progress of the game; (b) designing a set of IF-THEN rules that trigger utterance patterns based on current conditions in the game or conversation; (c) instrumentation of the game search process -- numbers of states evaluated, alpha/beta cutoffs, counts of reuses of hashed state values using Zobrist hashing, etc., as the basis for insightful comments on the game. Describe how you designed your utterance-related features in your Learning Diary entry. **Any of these approaches qualifies for the base Part II grade.** The LLM-based approach is listed separately as an extra-credit option because of the additional effort involved in designing effective prompts and handling API integration.

PART III: Written Component (60 points).

HTML Transcript (10 points)

Create an HTML transcript of a match between your agent and that of another person or partnership in the class. (Do not have your agent play against it self for this transcript, and do not have it play against an agent from the starter code.)

It's easy to create the transcript. Simply check that `USE_HTML` is set to `True` near the top of `Game_Master_Offline.py` (it is by default). Then whatever agents and game it runs will result in an HTML file being created. The title will be generated in a way that identifies the agents and the game type. Please convert the .html file to .pdf for submission.

`USE_JSON`, on the same lines, is also `True` by default, and writes a `.json` transcript of the same game. You do not need it for this assignment unless your instructor asks for it, but it is what makes a game replayable — boards, utterances and all — in the browser-based game master.

The game type for this transcript should be Five-in-a-Row On Seven-by-Seven Board with Corners Forbidden.

The transcript will be one of the inputs used in grading the dialog ability of your agent.

Running a Game

You will normally use the file `Game_Master_Offline.py` to run games between agents. `RandomPlayer.py` is a sample agent that plays randomly and is provided for you to test against. At the bottom of `Game_Master_Offline.py`, you'll see that the file is already set up to play your agent against our sample agent (as soon as you edit in the correct name of your agent file). You may change this to test different starting configurations as well as play against yourself or another agent. See the instructions in that file for making those changes.

Learning Diary Entry (60 points)

Complete the Learning Diary Entry template (included in the starter code) and submit it as **`Learning-Diary-Entry.pdf`**. The template guides you through all required written elements: your agent's design goals and persona; how the twin differs from the base agent; analysis of your alpha-beta implementation (including a hands-on leaf-count measurement and a conceptual explanation of why pruning is correct); your static evaluation design with a worked example from a real game run; dialog and utterance design; challenges and solutions encountered; gen-AI tool use and attribution; and documentation of any extra credit items completed.

Extra Credit Options

There are two ways to get extra credit in Part I and three ways to get extra credit in Part II. There are four more ways to obtain extra credit during the tournament: making the top 50 percent of agents, making it into the top 5, being judged the best agent in terms of playing the game, and being chosen as one of the five best in terms of dialog (described below). **Agents developed with gen-AI assistance are fully eligible for the tournament.** Tournament performance reflects the quality of your design decisions — choice of static evaluation features, search depth management, and prompting strategy — not the authorship of the code. Note that if there are too many tied games in any round using a particular time limit per move, then the round will be repeated with a shorter time limit, so that the stronger agents get a better chance to show their strength. On that last one: we will award 5 points of extra credit to 5 of the agents that, in the opinion of the staff, are the best in terms of dialog, with features such as relevance of comments to the game, including possible comments on technical issues during the search, clarity of persona, degree of apparent engagement in the dialog with the opponent, etc. Each item is worth 5 points, it's conceivable that some agent could not only win the tournament, but score in all the other extra credit, too, for a whopping 45 points of extra credit.

If you do any extra credit items, document them in Section 11 (Extra Credit) of your Learning Diary entry, identifying what options you completed and describing them.

in Part I:

- A. Implement the use of static evaluation of children to order the search, so as to increase the likelihood of alpha-beta cutoffs.

  Then figure out a way to measure the benefit of this technique over the default ordering, and document the results in Section 11 of your Learning Diary entry.

  Be sure to use a fixed-depth search in the comparisons, such as 4 ply.

- B. Implement Zobrist hashing. Also instrument your code so that the hashing statistics can be reported: Number of Writes to the hash table, number of read attempts, and number of successful attempts (those attempts where previously hashed date is found and returned).

  If you complete this option, document a brief description of Zobrist hashing and the benefits you measured in Section 11 of your Learning Diary entry. Note that saved values of states can be used for ordering children just like in A (above), to increase the likelihood of alpha-beta cutoffs.

in Part II:

- Use well thought-out prompting to get an LLM to come up with some or all of your agent's utterances, through a Python-callable API such as Google's Gemini. Note: Your agent code should, by default, not import any external APIs. However, in your **`prepare`** method, your agent should check to see if the current playing mode is DEMO, and if so, it may then import the Gen-AI api you are using. **Wrap that import, and every call you make through it, in a `try/except` that falls back to your non-LLM utterances.** Your agent must keep playing when the API is unavailable, and there are several ordinary situations in which it will be: the staff may not have your SDK installed, there may be no network, your quota may be exhausted, and the browser-based game master cannot reach any LLM endpoint at all (browsers block those requests, and the SDKs' synchronous HTTP does not work there). An agent that raises instead of falling back will not load, and will forfeit. Ideally, any LLM you use will be a standard one for which the staff can easily install the API to test your agent in DEMO mode. However, if you intend for the staff to be able to do this, you should hard-code your own API key into your agent, so that the staff does not need to set any environment variables just for your agent. If you do not wish to do that but wish to be considered for possible selection in the best utterances area, we may need to see more than one sample game transcript and we recommend submitting two of the game transcript PDF files (with different opponents) in that case.

  You may also use a locally hosted LLM if you have access to one. If the staff will not be able to easily use it, too, then provide one or more extra transcripts to show what your agent can do, as above.

- Implement a feature in your agent such that, if the opponent says "Tell me how you did that", your agent gives a thorough explanation of real statistics about its own computation of the last move, with counts of states evaluated statically, time spent, numbers of cutoffs, etc. Include an example in Section 11 of your Learning Diary entry where this feature comes into action.
- Implement a feature in your agent such that, if the opponent says "What's your take on the game so far?" then your agent will use the history of the game (past states and past utterances) to come up with a story about the game from the beginning to the current state, and a prediction of who will win. Include an example in Section 11 of your Learning Diary entry where this feature comes into action.

What to Turn In.

Turn in the following files:

- **`yourID_KInARow.py`** — your agent code
- GameTranscript**`.pdf`** — the game transcript generated by **`Game_Master_Offline.py`**, converted to PDF
- **`Learning-Diary-Entry.pdf`** — your completed Learning Diary entry

If you have completed extra credit items or wish to enter the competition, document these in Section 11 of your Learning Diary entry and submit any additional qualifying game transcripts alongside the files above.

For 1 point of participation credit, fill out the planning questionnaire anytime between Friday, Oct. 16 and Wednesday night (midnight) Oct. 21.

Updates and Corrections

If necessary, updates and corrections will be posted here and/or mentioned in class or on the course discussion board.